No-Code Toolkit for Unity XR - User Documentation Guide

1. Introduction

The No-Code Toolkit is a collection of reusable Unity scripts that allow users to create interactive behaviors without writing additional code for each interaction. The toolkit uses the Unity Inspector to configure triggers, timers, collisions, and sequences of actions.

To add a component from this toolkit to an object, find the script in the Project view and drag it to the object in either the Hierarchy or the Inspector (if the object is currently selected). Finally, select the object and find the script component in it's Inspector window to configure the events and timings.

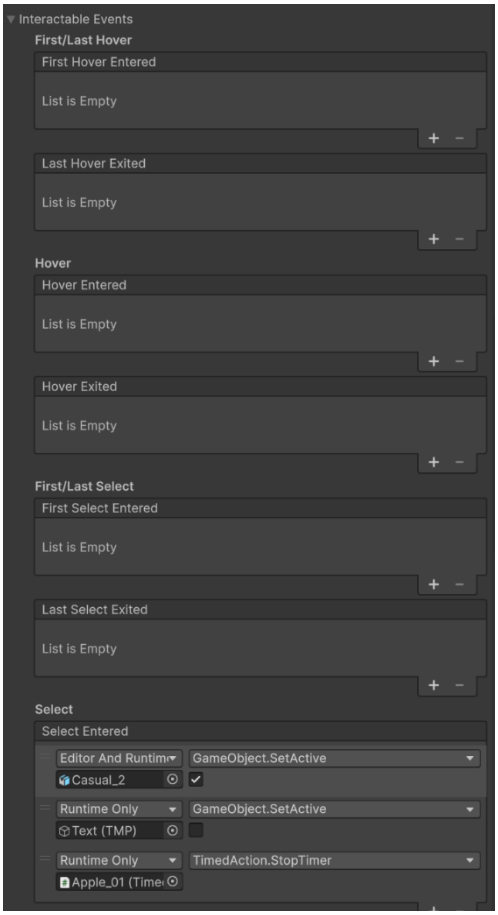
2. XR Grab Interactable

Unity's XR Grab Interactable component allows an XR user to grab and move an object using the controllers.

When should I use it?

Use XR Grab Interactable when an object should be picked up, moved, or thrown by the user.

How to set it up



1. Select the GameObject you want the user to grab.
2. Ensure the object has a Collider. If not, click "Add Component" in the Inspector and search for a collider component. Select an appropriate type and edit the collider if necessary.
3. Add the **XR Grab Interactable** component.
4. Enter Play mode and test grabbing the object.

You can connect grabbing to other behaviors by configuring other behaviors to the Interactable Events of the component.

On the left, we can see the Interactable Events list. By clicking the plus sign for each event, we can add behaviors which will be triggered by the event. For example, in the screenshot, The Select Entered event triggers several behaviors. The object called Casual_2 is set to active, the object called Text (TMP) is set to inactive and a timer is paused on the TimedAction component of Apple_01. These events all occur instantaneously and not in a sequence.

Below is a guide to the relevant events of the XR Grab Interactable.

Event	*Fires when...*
Hover Entered / Exited	*A hand or ray starts/stops pointing at the object*
Select Entered / Exited	*The object is grabbed / released*
Activated / Deactivated	*The object is held and the player presses/releases the trigger button*

3. OnTriggerAction

What does it do?

The OnTriggerAction component sets off an event when an object with a specified tag enters or exits a trigger collider. This is useful for creating proximity-based interactions, such as activating an object when the player approaches it.

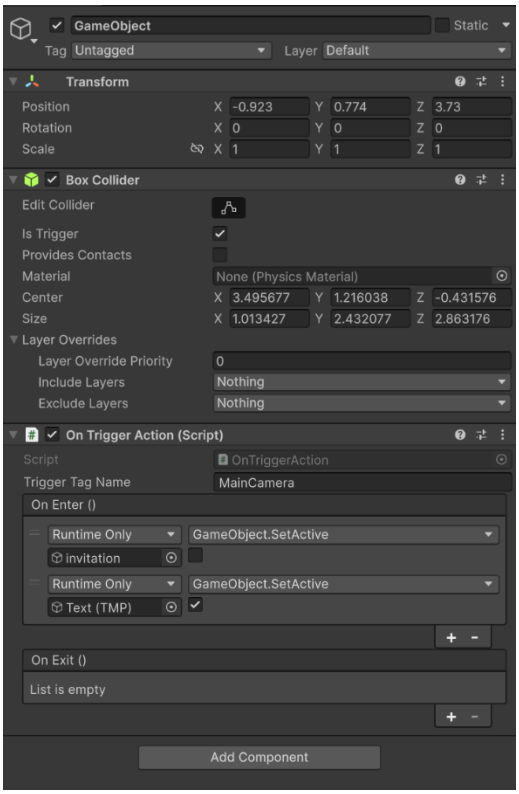
How to set it up

1. Select the GameObject that will act as the trigger.
2. Add a Collider, such as a Box Collider.
3. Enable **Is Trigger** on the Collider.
4. Add the OnTriggerAction component.
5. Set **Trigger Tag Name** to the tag of the object that should activate the trigger.
6. Expand **On Enter** and add the GameObject that should respond.
7. Select the function to execute when the tagged object enters the trigger.
8. Configure **On Exit** similarly if you want a different action when the object leaves.
5. Enter Play mode and move the tagged object into and out of the trigger area.

Settings

Setting	*Purpose*	*Default*
Trigger Tag Name	*Tag required for the entering or exiting object*	*MainCamera*
On Enter	*Actions invoked when the tagged object enters*	*Empty event*
On Exit	*Actions invoked when the tagged object exits*	*Empty event*

The tag must match an existing Unity tag exactly. Trigger events also require an appropriate Collider setup and, under Unity's physics rules, a Rigidbody on at least one of the interacting objects.



On the left is a screenshot of the Inspector view of a component that uses the On Trigger Action script. As you can see, the object has a Box Collider with the Is Trigger setting activated as well as the script component.

The Trigger Tag Name is set to “MainCamera” which means that the trigger will be activated when the player enters the collider. The On Enter() setting fires two behaviors: an object called “invitation” is deactivated and an object called “Text (TMP)” is activated in the scene. The On Exit() setting is empty, which just means that nothing will occur when the player exits the collider.

As you can see, you can edit the Trigger Tag Name to any combination of characters. However, in order for the script to work, it must be set to the exact, case-sensitive tag name of an object in your scene.

When selecting the targets of your events, you can drag objects directly from the Hierarchy into the boxes under the event dropdown which says “Runtime Only”.

5. TimedAction

What does it do?

The TimedAction component triggers a behavior after a specified duration. It can run automatically when the component starts or be started by another component or event. It can also be set to repeat the behavior, with the duration indicating the time interval between each repetition.

How to set it up

1. Select the GameObject that should contain the timer.
2. Add the TimedAction component.

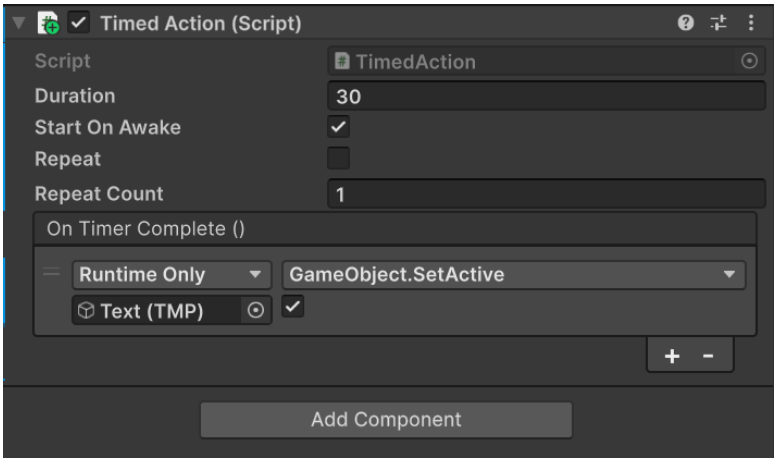
3. Set **Duration** to the number of seconds to wait.
4. Enable **Start On Awake** if the timer should begin automatically when the scene starts.
5. Configure **Repeat** if the action should happen more than once.
6. Set **Repeat Count** if you want to limit the number of additional repetitions.
7. Add an event to **On Timer Complete** and select the action that should happen when the timer finishes.
8. If automatic starting is disabled, connect a suitable event to the component's StartTimer() method.
9. Test the timer in Play mode.

Settings

Setting	***Purpose***	***Default***
Duration	*Number of seconds before the timer completes*	*5.0*
Start On Awake	*Starts the timer automatically when the component starts*	*false*
Repeat	*Repeats the action after the duration elapses*	*false*
Repeat Count	*Number of additional repetitions when repeating; zero means unlimited repetitions*	*0*
On Timer Complete	*Actions invoked when the timer finishes*	*Empty event*

When **Repeat** is disabled, the timer completes once. When **Repeat** is enabled, a Repeat Count of zero allows unlimited repetitions; a positive count limits the additional repetitions.

The StartTimer() method resets the timer and its repetition counter. The StopTimer() method stops it.



The image above shows a Timed Action component in the Unity Inspector. This specific component starts a 30 second timer as soon as the object activates and activates the object called “Text (TMP)” once the timer is done. This specific timer does not repeat.

6. CollideAction

What does it do?

The CollideAction component triggers a behavior when the object collides physically with another object that has a specified tag.

Unlike OnTriggerAction, this component responds to physical collisions rather than objects entering a trigger collider.

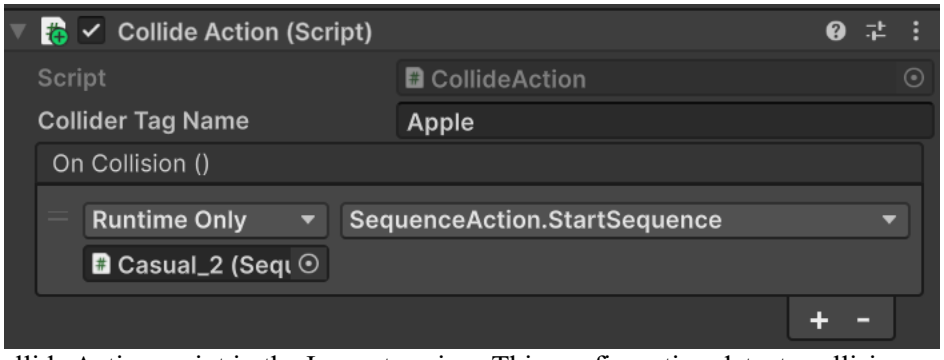
How to set it up

1. Select the GameObject that should detect a collision.
2. Ensure it has an appropriate Collider.
3. Add the CollideAction component.
4. Set **Collider Tag Name** to the tag of the object that should activate the event.
5. Expand **On Collision** and add the GameObject that should respond.
6. Select the behavior to activate when the collision occurs.
7. Enter Play mode and make the tagged object physically collide with the detecting object.

Settings

Setting	***Purpose***	***Default***
Collider Tag Name	*Tag required for the other object in the collision*	*MainCamera*
On Collision	*Actions invoked when the matching collision occurs*	*Empty event*

Make sure the tag exists and matches exactly. Also check that the Collider components and Rigidbody settings permit the collision to be detected.



Above is a Collide Action script in the Inspector view. This configuration detects collisions with an object tagged “Apple” and starts a sequence (see below) in the Casual_2 object. Notice that events in scripts can

be triggered by a different script. For example, a collision could start or stop a timer, a trigger can start or stop a sequence, etc. It is important that the scripts are associated with specific objects. You can reference a script in a different object by dragging that object into the target field (Casual_2 in the screenshot)of an event and selecting the script component in the dropdown on the right. (SequenceAction.StartSequence in the screenshot)

7. SequenceAction

What does it do?

The SequenceAction component triggers multiple events in a specified order. Each event can have an optional delay before it runs.

For example, a sequence could turn on a light immediately, play an animation after two seconds, and activate a second object after another delay.

How to set it up

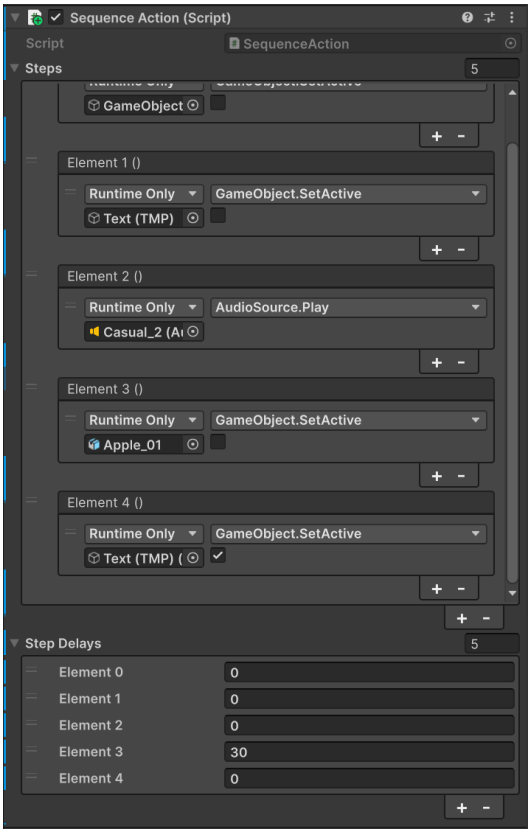
1. Select the GameObject that should control the sequence.
2. Add the SequenceAction component.
3. Expand **Steps** and set its size to the number of actions you want, or just start adding steps.
4. Configure each step's event with the object and function that should execute.
5. Expand **Step Delays** and enter the delay for each corresponding step. Use zero for a step that should execute without waiting.
6. Connect another component's event to StartSequence() if the sequence should begin in response to an interaction.
7. Enter Play mode and verify that the actions occur in the intended order.

Settings

Setting	***Purpose***	***Default***
Steps	*Ordered events to execute*	*Empty array*
Step Delays	*Delay associated with each step, in seconds*	*Empty array*

If a delay is missing for a step, it is treated as zero. Negative delays are also treated as zero. The first step executes immediately when the sequence starts if its delay is zero.

The StartSequence() method restarts the sequence from the beginning. The StopSequence() method stops the sequence.



On the left is a screenshot of a Sequence Action component in the Inspector view. The list of Steps details the events which will be triggered, in order, and the list of Step Delays details the delay between the start of the previous step and the start of the current step. The first step is triggered immediately after the first delay.

In the example shown, five total events occur. Element 0 represents the first step and it deactivates GameObject. The next steps activate and deactivate objects and trigger an audio source. Timers can also be started or stopped by sequence steps.

8. Connecting Components

The main advantage of the toolkit is that components can work together. A UnityEvent from one component can start another component's behavior.

For example, to start a timer when an object is grabbed:

1. Add XR Grab Interactable to the object.
2. Add TimedAction to the object or another suitable GameObject.
3. Disable Start On Awake so the timer does not start automatically.
4. Configure the timer's duration and completion event.
5. Use an appropriate XR selection event to invoke StartTimer().
6. Configure On Timer Complete to activate a light or another GameObject.
7. Test the entire interaction in Play mode.

Another example is to use OnTriggerAction to invoke StartSequence(). When the tagged object enters the trigger, the sequence begins and runs its configured steps. Reference the earlier screenshots (especially the Collide Action section) for Inspector view examples of these scripts working together.

Remember: A component will not automatically know which other object or function it should activate. You must assign the target object and function in the event field.